

Depth-First Search (DFS)

```
const int N = 1e5 + 5;
vector<int> adj[N];
vector<bool> visited(N);

void dfs(int node) {
    visited[node] = true;
    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor);
        }
    }
}
```

Breadth-First Search (BFS)

```
const int N = 1e5 + 5;
vector<int> adj[N];
vector<bool> visited(N);

void bfs(int start) {
    queue<int> q;
    q.push(start);
    visited[start] = true;

    while (!q.empty()) {
        int node = q.front();
        q.pop();
        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

Dijkstra's Algorithm (Shortest Path from 1 source)

```

const int INF = 1e9;
const int N = 1e5 + 5;
vector<pair<int, int>> adj[N]; // {neighbor, weight}
vector<int> dist(N, INF);

void dijkstra(int start) {
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    dist[start] = 0;
    pq.push({0, start}); // {distance, node}

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

```

Bellman-Ford Algorithm – $O(n \times m)$

```

const int INF = 1e9;
struct Edge {
    int u, v, w;
};

int main() {
    int n, m, src;
    vector<Edge> edges(m);

    vector<int> dist(n + 1, INF);
    dist[src] = 0;

    // Relax all edges n - 1 times
    for (int i = 1; i < n; ++i) {
        for (auto [u, v, w] : edges) {
            if (dist[u] != INF && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }

    // Check for negative weight cycle
    bool negativeCycle = false;

```

```

    for (auto [u, v, w] : edges) {
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            negativeCycle = true;
            break;
        }
    }
}

```

Floyd-Warshall Algorithm – $O(n^3)$

```

const int INF = 1e9;
const int N = 505;
int dist[N][N];

int main() {
    int n, m;
    // Initialize distances
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j)
            dist[i][j] = (i == j ? 0 : INF);

    // Read edges
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        dist[u][v] = min(dist[u][v], w); // in case of multiple edges
    }

    // Floyd-Warshall
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (dist[i][k] < INF && dist[k][j] < INF)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
}

```

Update Bellman-Ford After Adding One Edge $o(n)$:

```

void updateBellmanFord(int u, int v, int w, vector<int>& dist, const
vector<vector<pair<int, int>>>& adj) {
    if (dist[u] + w >= dist[v]) return; // No update needed

    dist[v] = dist[u] + w;
}

```

```

queue<int> q;
q.push(v);

while (!q.empty()) {
    int node = q.front(); q.pop();
    for (auto [nbr, cost] : adj[node]) {
        if (dist[node] + cost < dist[nbr]) {
            dist[nbr] = dist[node] + cost;
            q.push(nbr);
        }
    }
}

```

Update Floyd-Warshall After Adding Edge (u, v, w) O(n²) :

```

void updateFloyd(int u, int v, int w, vector<vector<int>>& dist, int n) {
    if (dist[u][v] <= w) return; // No update needed

    dist[u][v] = w;

    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j)
            if (dist[i][u] < 1e9 && dist[v][j] < 1e9)
                dist[i][j] = min(dist[i][j], dist[i][u] + w + dist[v][j]);
}

```

LCA

```

vector<int>dis(n);
vector<vector<int>>anc(20, vector<int>(n)) ;
function<void(int,int)>dfs=[&](int u, int p ){
    anc[0][u] = p ;
    for(auto f : adj[u]){
        if(f!=p){
            dis[f] = dis[u]+1;
            dfs(f,u);
        }
    }
};

```

```

for (int mask = 1; mask < 18; mask++)
{
    for (int u = 1; u <= n; u++)
    {
        anc[mask][u] = anc[mask-1][anc[mask-1][u]];
        // for max edge in path
        mx [mask][u] = max(mx[mask-1][u] , mx[mask-1][anc[mask-1][u]]) ;
    }
}

auto get_kth_ancestor=[&](int u, int k) {
    for (int i = 0; i < 20; ++i) {
        if (k & (1 << i)) {
            u = anc[i][u];
            if (u == 0) break;
        }
    }
    return u;
};

int getLCA(int u, int v)
{
    if (dis[v] > dis[u]) swap(u, v);

    int diff = dis[u] - dis[v];
    for (int i = 0; diff > 0; ++i) {
        if (diff & 1)
            u = anc[i][u];

        diff >>= 1;
    }

    if (u == v) return u;

    for (int i = anc.size() - 1; i >= 0; --i) {
        if (anc[i][u] != anc[i][v]) {
            u = anc[i][u];
            v = anc[i][v];
        }
    }
    return anc[0][u];
}

```

Flat Tree:

```

int timer = 0 ;
vector<int>newVals(n) ;
function<void(int,int)>dfs = [&](int u , int p){

```

```

    in[u] = timer++;
    for(auto f : adj[u]){
        if(f!=p)dfs(f,u);
    }
    out[u] = timer-1;
};
dfs(1,0);
for (int i = 1; i <= n; i++)
{
    newVals[in[i]] = arr[i-1];
}

```

Flat Tree For Mo Path:

```

function<void(int,int)>dfs=[&](int u , int p){
    anc[0][u] = p ;
    in[u] = timer++;
    for(auto f : adj[u]){
        if(f!=p){
            dis[f] = dis[u]+1;
            dfs(f,u);
        }
    }
    out[u] = timer++;
};
vi vals(n*2) , nodes(n*2);
for (int i = 1; i <= n; i++)
{
    vals[in[i]] = arr[i-1] ;
    vals[out[i]] = arr[i-1];
    nodes[in[i]] = i ;
    nodes[out[i]] = i ;
}

```

DSU:

```

struct Dsu
{
    vector<int> par , sz ;
    Dsu (int n){
        for (int i = 0; i <= n; i++)
        {
            sz.push_back(1);
            par.push_back(i);
        }
    }
}

```

```

    }
}
int find (int u){
    if(par[u]==u){
        return u ;
    }
    return par[u] = find(par[u]);
}

bool merge (int u , int v){
    u = find(u) , v = find(v) ;

    if(u==v)return false ;

    if(sz[v] > sz[u])swap(u,v);
    par[v] = u ;
    sz[u]+=sz[v];
    return true ;
}
};

```

DSU With Roll Back:

```

struct DSU {
    vector<int> par, sz, checkPoints;
    vector<pair<int, int>> updates;

    void init(int n) {
        par.assign(n + 1, {});
        iota(par.begin(), par.end(), 0);
        sz.assign(n + 1, 1);
        checkPoints.clear();
        updates.clear();
    }

    int find(int u) {
        while (u ^ par[u]) {
            u = par[u];
        }
        return u;
    }

    bool merge(int u, int v) {
        u = find(u), v = find(v);
        if (u == v) {
            return false;
        }
        if (sz[u] < sz[v]) {
            swap(u, v);

```

```

    }
    updates.emplace_back(v, u);
    sz[u] += sz[v];
    par[v] = u;
    return true;
}

void snapshot() {
    checkPoints.emplace_back(updates.size());
}

void rollback() {
    if (checkPoints.empty()) {
        return;
    }
    while (checkPoints.back() != updates.size()) {
        auto [v, u] = updates.back();
        updates.pop_back();
        sz[u] -= sz[v];
        par[v] = v;
    }
    checkPoints.pop_back();
}

int size(int u) {
    return sz[find(u)];
}

int sameSet(int u, int v) {
    return find(u) == find(v);
}

};

```

MO on Path On Tree :

```

void solve(){
    int n , q , timer = 0 , id = 1; cin>>n>>q ;
    vvi adj(n+1) , anc(20,vi(n+1));
    vi dis (n+1) , in(n+1) , out(n+1) , cnt (n+5) , res(q) , vals(n*2) , nodes(n*2);
    vector<bool> vis(n+1);
    map<int,int>mp;
    int arr[n] ;

    for (int i = 0; i < n; i++)
    {
        cin>>arr[i] ;
        if(mp[arr[i]]==0){
            mp[arr[i]] = id++ ;
        }
    }
}

```



```

    arr[i] = mp[arr[i]] ;
}
for (int i = 0; i < n-1; i++)
{
    int a , b ; cin>>a>>b;
    adj[a].push_back(b);
    adj[b].push_back(a);
}
function<void(int,int)>dfs=[&](int u , int p){
    anc[0][u] = p ;
    in[u] = timer++;
    for(auto f : adj[u]){
        if(f!=p){
            dis[f] = dis[u]+1;
            dfs(f,u);
        }
    }
    out[u] = timer++;
};
dfs(1,0);
vector<array<int,4>>qi ;
for (int i = 0; i < q; i++)
{
    int u , v ; cin>>u>>v;
    if(in[u]>in[v])swap(u,v) ; // Let ST(u) ≤ ST(v)
    int lc = getLCA(u,v) ;
    if(lc == u){
        qi.push_back({in[u],in[v],i,0}) ; // [ST(u), ST(v)]
    }else{
        qi.push_back({out[u],in[v],i,getLCA(u,v)}) ; // [EN(u), ST(v)] + [ST(P),
ST(P)] .
    }
}
int BS = sqrt(2*n)+1;
sort(all(qi),[&](array<int,4>q1,array<int,4>q2){
    return make_pair(q1[0]/BS , q1[1]) < make_pair(q2[0]/BS , q2[1]);
});
for (int i = 1; i <= n; i++)
{
    vals[in[i]] = arr[i-1] ;
    vals[out[i]] = arr[i-1];
    nodes[in[i]] = i ;
    nodes[out[i]] = i ;
}
int ans = 0 ;

auto add = [&](int ind){
    int val = vals[ind] ;
    if(!cnt[val])ans++;
    cnt[val]++;
};
auto ers = [&](int ind){
    int val = vals[ind] ;
    cnt[val]--;
};

```

```

        if(!cnt[val])ans--;
    };
    auto go = [&](int ind){
        int u = nodes[ind];
        vis[u] = vis[u]^1 ;
        if(vis[u])add(ind);
        else ers(ind);
    };
    int l = qi[0][0];
    int r = 1 ;
    go(l) ;
    for (auto [lx , rx , ix , lc] : qi)
    {
        while (lx<l)go(--l);
        while (rx>r)go(++r);

        while (lx>l)go(l++);
        while (rx<r)go(r--);

        if(lc)add(in[lc]) ;

        res[ix] = ans ;

        if(lc)ers(in[lc]) ;

    }
    for (int i = 0; i < q; i++)cout<<res[i] ln ;
}

```

BFS 01 :

```

void bfs01(int src) {
    const int n = adj.size();
    const int INF = 1e9;
    vector<int> d(n, INF);
    d[src] = 0;
    deque<int> q;
    q.push_front(src);
    while (!q.empty()) {
        int v = q.front();
        q.pop_front();
        for (auto edge: adj[v]) {
            int u = edge.first;
            int w = edge.second;
            if (d[v] + w < d[u]) {
                d[u] = d[v] + w;
                if (w == 1)
                    q.push_back(u);
            }
        }
    }
}

```

```

        else
            q.push_front(u);
    }
}
}
}

```

LCA in $O(1)$:

```

vector<vector<int>> adj;
vector<int> in, lvl;
vector<vector<int>> euler;

void dfs0(int u, int par) {
    in[u] = euler[0].size();
    euler[0].emplace_back(u);
    for (auto &v: adj[u]) {
        if (par == v) continue;
        lvl[v] = lvl[u] + 1;
        dfs0(v, u);
        euler[0].emplace_back(u);
    }
}

int mrg(int u, int v) {
    return lvl[u] < lvl[v] ? u : v;
}

void precompute_lca(int root = 0) {
    euler = {{}};
    in.assign(adj.size(), {});
    lvl.assign(adj.size(), {});
    dfs0(root, 0);
    const int sz = euler[0].size();
    for (int msk = 1; (1 << msk) <= sz; ++msk) {
        euler.emplace_back(sz - (1 << msk) + 1);
        for (int i = 0; i + (1 << msk) <= sz; ++i) {
            euler[msk][i] = mrg(
                euler[msk - 1][i],
                euler[msk - 1][i + (1 << (msk - 1))]
            );
        }
    }
}

```

Small to Large Tree :

```

const int N = 3e5 + 5;
vector<vector<int>> adj;
int in[N], sz[N], tree[N];
int t; /// reset this to 0 each testcase

void dfs0(int u, int p = -1) {
    tree[t] = u, in[u] = t++;
    sz[u] = 1;
    for (int v: adj[u]) {
        if (v == p) continue;
        dfs0(v, u);
        sz[u] += sz[v];
    }
}

void add(int node) {}
void rem(int node) {}

void dfs1(int u, int p, bool keep) {
    int bg = -1, mx = -1;
    for (int &v: adj[u]) {
        if (v != p && sz[v] > mx)
            mx = v, bg = v;
    }
    for (int &v: adj[u]) {
        if (v == p or v == bg) continue;
        dfs1(v, u, false);
    }
    if (~bg) {
        dfs1(bg, u, true);
    }
    add(u);
    for (int v: adj[u]) {
        if (v == p or v == bg) continue;
        for (int i = 0; i < sz[v]; ++i) {
            int x = tree[in[v] + i];
            add(x);
        }
    }
}

///// get the answers here

if (!keep) {
    for (int i = 0; i < sz[u]; ++i) {
        int x = tree[in[u] + i];
        rem(x);
    }
}
}

```

HLD On Tree :

```

struct HLD
{
    vi big , head , siz , par , id , dpth;
    int timer ;
    HLD(int n , vvi&adj): big(n+1) ,par(n+1), id(n+1), head(n+1), dpth(n+1),
    siz(n+1,1){
        dfs(1 , adj);
        timer = 0 ;
        head[1] = 1 ;
        dfs1(1 , adj);
    }
    void dfs (int u , vvi& adj){
        for(auto f : adj[u]){
            if(f == par[u])continue;
            par[f] = u ;
            dpth[f] = dpth[u]+1 ;
            dfs(f,adj);
            siz[u]+=siz[f] ;
            if(!big[u] || siz[f] > siz[big[u]]) big[u] = f ;
        }
    }
    void dfs1 (int u , vvi& adj){
        id[u] = timer++;
        if(big[u]) head[big[u]] = head[u] ,dfs1(big[u] , adj) ;
        for(auto f : adj[u]){
            if(f == par[u] || f == big[u])continue;
            head[f] = f ;
            dfs1(f , adj);
        }
    }
    vector<pair<int,int>>path(int u , int v){
        vector<pair<int,int>>res ;
        while (true)
        {
            if(head[u] == head[v]){
                if(dpth[u] > dpth[v])swap(u,v);
                res.push_back({id[u] , id[v]});
                return res ;
            }
            if(dpth[head[u]] > dpth[head[v]])swap(u,v);
            res.push_back({id[head[v]],id[v]});
            v = par[head[v]];
        }
    }
};

```

HLD sorted Path :

```
vector<pair<int,int>> path(int u , int v){
    vector<pair<int,int>> left, right;

    while(head[u] != head[v]) {
        if(dpth[head[u]] > dpth[head[v]]) {
            // segment من u لحد head[u]
            left.push_back({id[head[u]], id[u]});
            u = par[head[u]];
        } else {
            // segment من v لحد head[v]
            right.push_back({id[head[v]], id[v]});
            v = par[head[v]];
        }
    }

    // في نفس السلسلة v و u دلوقتى
    if(dpth[u] > dpth[v]) swap(u,v);
    // segment الأخير من u لحد v
    left.push_back({id[u], id[v]});

    // علشان يطلع من left قلب u → (LCA) → v
    reverse(left.begin(), left.end());

    // ندمج
    left.insert(left.end(), right.begin(), right.end());
    return left;
}
```

Sack in tree

```
auto collect = [&](int u , int d){
    if(freq[arr[u]]==0)ans++;
    freq[arr[u]]+=d;
    if(freq[arr[u]]==0)ans--;
};

function<void(int,int,int)> addSubtree = [&](int u, int p, int val){
    collect(u, val);
    for(auto f : adj[u]){
        if(f == p) continue;
        addSubtree(f, u, val);
    }
};
```

```
function<void(int,int)>dfs0=[&](int u ,int p){
    siz[u] = 1 ;
    for(auto f : adj[u]){
        if(f == p)continue;
        dfs0(f , u);
        siz[u] += siz[f];
        if(!big[u] || siz[big[u]] < siz[f])big[u] = f ;
    }
};
function<void(int,int,bool)>dfs1=[&](int u ,int p , bool keep){
    for(auto f : adj[u]){
        if(f == p || f == big[u])continue;
        dfs1(f , u , false);
    }
    if(big[u]) dfs1(big[u] , u , true);

    collect(u , 1);
    for(auto f : adj[u]){
        if(f == p || f == big[u])continue;
        addSubtree(f , u , 1);
    }
    res[u] = ans ;
    if(!keep)addSubtree(u , p , -1);
};
dfs0(1,1);
dfs1(1,1,true);
```